DS314: GENERATIVE AI & LLM SYSTEMS · ACADEMIC LAB MODULE

DS314 - Production Engine: Module 01 Foundation Inference Engine & Decoding Thermodynamics

Format:	Jupyter Notebook (.ipynb)	**Source:**	Module_01_Foundation_Engine.ipynb	**Render Engine:**	KaTeX + MathML (Zero-Delim Math)
Academic Level:	3rd Year BS Data Science				

Module 1: Foundation Inference Engine, Decoding Thermodynamics & Tokenomics

DS314: Generative AI & Large Language Model Engineering — Production Build Series

Executive Overview & Stage Objective

In this foundational stage, you will architect the bedrock component of our enterprise chatbot: the `FoundationModelClient`.

Rather than directly making ad-hoc API calls throughout your application, production GenAI systems decouple model interaction into an **isolated, resilient, and vendor-agnostic engine**. This engine enforces strict hyperparameter boundaries, protects against secret leaks, intercepts transient network/quota faults via exponential backoff with jitter, and normalizes disparate model outputs into a typed, structured payload.

Key Deliverables in this Notebook:

1. `ModelConfig`: A strongly-typed configuration container with boundary validation.
2. `LLMResult`: A normalized return container encapsulating generation text and token economics.
3. `FoundationModelClient`: A fault-tolerant wrapper around Google Gemini (`ChatGoogleGenerativeAI`).
4. Self-Assessment Checkpoint: 3 oral exam defense questions.

1. Concept Reviewer (The "Why")

1.1 Autoregressive Generation & The Logits Layer

Large Language Models are probabilistic next-token predictors. Given an input context $X = (x_1, \ldots, x_n)$, the transformer maps internal activations through an unembedding projection layer to generate unnormalized log-probability values called **logits** $\mathbf{z} \in \mathbb{R}^{|\mathcal{V}|}$ over vocabulary $\mathcal{V}$.

1.2 Decoding Thermodynamics

To convert raw logits into next-token probabilities $P(x_{n+1} = w_i)$, the model applies the **Softmax function scaled by temperature** τ:

$$P(w_i) = \frac{\exp(z_i / \tau)}{\sum_{j=1}^{|\mathcal{V}|} \exp(z_j / \tau)}$$

- **Low Temperature ($\tau \to 0$):** Sharpens the distribution. The token with the highest logit dominates exponentially, approaching deterministic (greedy) argmax selection. Best for factual extraction, RAG reasoning, and JSON formatting.
- **High Temperature ($\tau > 1.0$):** Compresses differences between logits, flattening the distribution toward uniform randomness. Boosts lexical diversity and creative hallucination.
- **Top-p (Nucleus Sampling):** Dynamically restricts candidates to the smallest subset of tokens whose cumulative probability exceeds p.

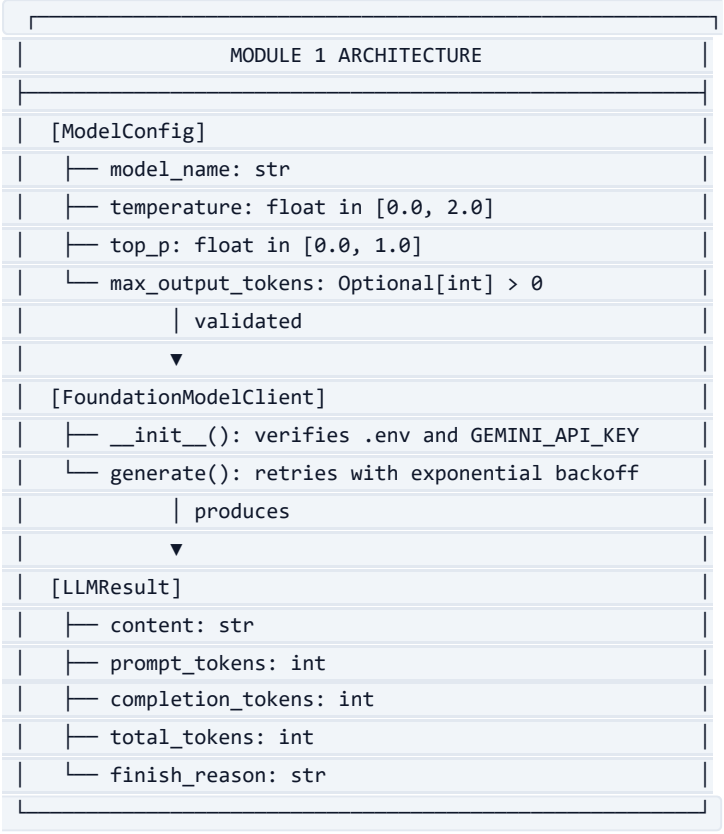
1.3 Prefill vs. Generation Compute Asymmetry

- **Prefill Phase (Prompt Processing):** Compute-bound. All prompt tokens are evaluated in parallel on GPU tensor cores to populate the Key-Value (KV) cache.
- **Generation Phase (Output Generation):** Memory-bandwidth bound. Each output token is generated one-by-one sequentially, requiring an entire forward pass per token. Latency is dominated by output token count, not input prompt length.

1.4 Production Failure Modes Guarded by this Module

1. **HTTP 429 Quota Exhaustion (`ResourceExhausted`):** Unthrottled calls crash when hitting Requests Per Minute (RPM) thresholds. Production clients must apply exponential backoff with randomized jitter.
2. **The `AIMessage` Trap:** LangChain returns an `AIMessage` object. Downstream components expecting standard strings crash with `AttributeError` if properties are not extracted cleanly.
3. **Secret Leakage:** Committing raw API keys violates security compliance. Secrets must be loaded exclusively at runtime via environment variables.

2. Technical Blueprint: System Interfaces



In [ ]:

```python
# Cell 1: Environment Setup & Package Sanity Check
import os
import time
import random
from typing import Optional, Dict, Any
from dataclasses import dataclass
from dotenv import load_dotenv

# Verify LangChain and Google GenAI imports
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.messages import HumanMessage, BaseMessage

print('Environment initialized successfully. Ready to build!')
```

3. Implementation Challenge

Task 1: ModelConfig & LLMResult Data Structures

Implement the validation logic inside `ModelConfig.validate()`:

- `temperature` must satisfy $0.0 \leq \tau \leq 2.0$.
- `top_p` must satisfy $0.0 \leq p \leq 1.0$.
- `max_output_tokens` (if provided) must be an integer > 0.
- If any condition fails, raise a `ValueError` with a clear diagnostic explanation.

In [ ]:

```python
@dataclass
class ModelConfig:
    model_name: str = "gemini-2.5-flash"
    temperature: float = 0.2
    top_p: float = 0.95
    max_output_tokens: Optional[int] = 1024

    def validate(self) -> None:
        """
        Validates configuration boundaries.
        Raises ValueError if any parameter is out of bounds.
        """
        # TODO: Check if temperature is between 0.0 and 2.0 (inclusive)
        # If not, raise ValueError

        # TODO: Check if top_p is between 0.0 and 1.0 (inclusive)
        # If not, raise ValueError

        # TODO: If max_output_tokens is not None, verify it is an int > 0
        # If not, raise ValueError
        pass


@dataclass
class LLMResult:
    """Normalized, vendor-agnostic return structure."""
    content: str
    prompt_tokens: int
    completion_tokens: int
    total_tokens: int
    finish_reason: str
    model_name: str
```

In [ ]:

```python
# Test Cell 1: Validate ModelConfig boundaries
print("Running validation tests on ModelConfig...")

# 1. Valid configuration test
valid_config = ModelConfig(temperature=0.7, top_p=0.9, max_output_tokens=500)
valid_config.validate()

# 2. Invalid temperature test
try:
    bad_temp = ModelConfig(temperature=2.5)
    bad_temp.validate()
    print("FAIL: ModelConfig accepted temperature > 2.0")
except ValueError:
    print("PASS: Temperature validation caught out-of-bounds value.")

# 3. Invalid top_p test
try:
    bad_p = ModelConfig(top_p=-0.1)
    bad_p.validate()
    print("FAIL: ModelConfig accepted top_p < 0.0")
except ValueError:
    print("PASS: Top-p validation caught negative value.")

# 4. Invalid max_output_tokens test
try:
    bad_tokens = ModelConfig(max_output_tokens=0)
    bad_tokens.validate()
    print("FAIL: ModelConfig accepted max_output_tokens <= 0")
except ValueError:
    print("PASS: Token limit validation caught zero limit.")
```

Task 2: Fault-Tolerant FoundationModelClient

Implement the `FoundationModelClient` class with:

1. **Secret Hygiene:** Call `load_dotenv()` and verify `GEMINI_API_KEY` exists in `os.environ`. If absent, raise `ValueError`.
2. **Model Instantiation:** Configure `ChatGoogleGenerativeAI` with the parameters from `self.config`.
3. **Exponential Backoff with Jitter:** In `generate()`, wrap `.invoke([HumanMessage(content=prompt)])` in a retry loop (minimum 3 attempts).

$$\text{delay} = \text{base_delay} \times 2^{\text{attempt}} + \text{random.uniform}(0, 0.5)$$

4. **Safe Metadata Extraction:** Extract `.content` as a string, extract token counts (`prompt_tokens`, `completion_tokens`, `total_tokens`) from `response.usage_metadata` or fallback to `response.response_metadata`, extract the `finish_reason`, and return an `LLMResult` object.

In [ ]:

```python
class FoundationModelClient:
    def __init__(self, config: Optional[ModelConfig] = None):
        """
        Initializes environment variables, validates secrets, and instantiates the model.
        """
        # TODO: Load .env using load_dotenv()

        # TODO: Verify GEMINI_API_KEY exists in os.environ
        # If missing or blank, raise ValueError("GEMINI_API_KEY is not set in environment or .env file.

        # Set and validate configuration
        self.config = config or ModelConfig()
        self.config.validate()

        # Initialize underlying LangChain Google GenAI model
        self.llm = self._init_model()

    def _init_model(self) -> ChatGoogleGenerativeAI:
        """
        Instantiates ChatGoogleGenerativeAI with config hyperparameters.
        """
        # TODO: Instantiate and return ChatGoogleGenerativeAI(
        #     model=self.config.model_name,
        #     temperature=self.config.temperature,
        #     top_p=self.config.top_p,
        #     max_output_tokens=self.config.max_output_tokens
        # )
        pass

    def generate(
        self,
        prompt: str,
        max_retries: int = 3,
        base_delay: float = 1.0
    ) -> LLMResult:
        """
        Executes inference with exponential backoff and jitter.
        Extracts content and token usage into a clean LLMResult container.
        """
        # TODO: Implement retry loop over range(max_retries)
        # for attempt in range(max_retries):
        #     try:
        #         1. Invoke model: response = self.llm.invoke([HumanMessage(content=prompt)])
        #         2. Extract text: content_text = str(response.content)
        #         3. Extract tokens safely from response.usage_metadata:
        #            usage = getattr(response, "usage_metadata", {}) or {}
        #            p_tokens = usage.get("input_tokens", 0)
        #            c_tokens = usage.get("output_tokens", 0)
        #            t_tokens = usage.get("total_tokens", p_tokens + c_tokens)
        #         4. Extract finish reason: inspect response.response_metadata
        #         5. Return LLMResult(...)
        #     except Exception as e:
        #         if attempt < max_retries - 1:
        #             delay = (base_delay * (2 ** attempt)) + random.uniform(0, 0.5)
        #             print(f"[Retry {attempt+1}/{max_retries}] Exception: {e}. Backing off for {delay:
        #             time.sleep(delay)
        #         else:
        #             raise e
        pass
```

In [ ]:

```python
# Test Cell 2: Live Inference Verification
# Complete the TODO stubs in the cell above before running this!

try:
    test_config = ModelConfig(
        model_name="gemini-2.5-flash",
        temperature=0.1,
        max_output_tokens=150
    )
    client = FoundationModelClient(config=test_config)

    prompt = "In precisely 2 sentences, explain the mathematical function of temperature in the Softmax
    print(f"Executing inference with prompt: {prompt}\n")

    result = client.generate(prompt)

    print("--- INFERENCE SUCCESSFUL ---")
    print("Response Content:\n", result.content)
    print("\n--- TOKEN USAGE AND METADATA ---")
    print(f"Prompt Tokens:     {result.prompt_tokens}")
    print(f"Completion Tokens: {result.completion_tokens}")
    print(f"Total Tokens:      {result.total_tokens}")
    print(f"Finish Reason:     {result.finish_reason}")
    print(f"Model Name:        {result.model_name}")

    # Programmatic Assertions
    assert isinstance(result.content, str) and len(result.content) > 10, "Content must be non-empty str
    assert result.total_tokens > 0, "Total tokens must be greater than zero!"
    assert isinstance(result, LLMResult), "Return value must be an instance of LLMResult!"
    print("\n[PASSED] All programmatic validation criteria met!")
except Exception as err:
    print(f"\n[TEST FAILED]: {err}")
```

4. Self-Assessment Checkpoint (Oral Exam / Interview Defense)

Before moving to Module 2, answer these 3 diagnostic defense questions directly in the markdown cells below:

Question 1: Thermodynamic Limits

Mathematically, what happens to the Softmax probability distribution when $\tau \to 0$ versus $\tau \to \infty$? Why does $\tau = 0$ NOT guarantee 100% bit-exact output across different GPU clusters?

Your Answer:

> *[Write your answer here]*

Question 2: Compute Asymmetry (Prefill vs. Decode)

Why does processing a 2,000-token prompt take drastically less time than generating a 2,000-token response on modern GPU architectures? Explain the difference between tensor-core parallelization and memory-bandwidth bound autoregression.

Your Answer:

> *[Write your answer here]*

Question 3: Fault Tolerance & The Thundering Herd

What is the "Thundering Herd" problem in distributed microservices, and why is randomized jitter mathematically necessary when implementing exponential backoff for API rate-limit resilience?

Your Answer:

> *[Write your answer here]*